

AI TRAINING – WEEK 2

DAY 3 ASSIGNMENT

File I/O, CSV, JSON, Exceptions, Modules & Packages

Read → Clean → Handle Errors → Export

1. What You Will Learn

You will learn how Python works with files and structured data. You will read CSV and JSON files, clean records, handle bad input safely, and export the results.

- File I/O – read and write files
- CSV – read tabular data
- JSON – read and write structured data
- Exceptions – handle errors using try/except
- Modules – reuse Python code from another file
- Packages – understand how related modules are organized
- Clean and validate data
- Export processed results

2. Student Deliverable – Keep It Simple

For this assignment, your main submission is ONE data processing script.

Main file: data_processing_YourName.py

Keep the sample CSV/JSON files used by your program in the same project folder.

- Read input data
- Clean the records
- Handle bad or missing data
- Process the data
- Export the cleaned result
- Use functions to keep the code organized

No screenshots or separate Word/PDF submission is required.

3. Part A – File I/O Basics

Create notes.txt using Python.

- Write three lines into the file
- Read the file and print its contents
- Append one more line.
- Read the file again.
- Use the with statement when possible.

Remember: r = read, w = write, a = append.

CODE:

1. Write three lines into the file

with open("notes.txt", "w") as file:

```
    file.write("i am yasaswini.\n")
```

```
    file.write("i am from nellore.\n")
```

```
    file.write("i am learning python.\n")
```

2. Read the file and print its contents

with open("notes.txt", "r") as file:

```
    content = file.read()
```

```
    print("file contents:")
```

```
    print(content)
```

3. Append one more line

with open("notes.txt", "a") as file:

```
    file.write("i am studying btech final year.\n")
```

4. Read the file again

with open("notes.txt", "r") as file:

```
    content = file.read()
```

```
    print("file contents after appending:")
```

```
    print(content)
```

OUTPUT:

file contents:

i am yasaswini.

i am from nellore.

i am learning python.

file contents after appending:

i am yasaswini.

i am from nellore.

i am learning python.

i am studying btech final year.

4. Part B – Read a CSV File

Create students.csv with:

```
name,age,marks,city
Rahul,21,85,Hyderabad
Priya,22,92,Bangalore
Arjun,20,78,Chennai
```

- Open the CSV file.
- Read each record.
- Print each student's name and marks.
- Find the average marks.
- Find the student with the highest marks.

CODE:

```
import csv
```

```
# 1. Create students.csv
```

```
with open("students.csv", "w", newline="") as file:
```

```
    writer = csv.writer(file)
```

```
    writer.writerow(["name", "age", "marks", "city"])
```

```
    writer.writerow(["Rahul", 21, 85, "Hyderabad"])
```

```
    writer.writerow(["Priya", 22, 92, "Bangalore"])
```

```
    writer.writerow(["Arjun", 20, 78, "Chennai"])
```

```
# 2. Open and read the CSV file
```

```
with open("students.csv", "r") as file:
```

```
    reader = csv.DictReader(file)
```

```
    total_marks = 0
```

```
    count = 0
```

```
highest_marks = 0
highest_student = ""
for student in reader:
    name = student["name"]
    marks = int(student["marks"])
    print("Name:", name)
    print("Marks:", marks)
    total_marks = total_marks + marks
    count = count + 1
    if marks > highest_marks:
        highest_marks = marks
        highest_student = name
# 3. Find average marks
average = total_marks / count
print("Average marks:", average)
print("Highest marks:", highest_marks)
print("Highest marks student:", highest_student)
```

OUTPUT:

Name: Rahul

Marks: 85

Name: Priya

Marks: 92

Name: Arjun

Marks: 78

Average marks: 85.0

Highest marks: 92

Highest marks student: Priya

5. Part C – Read JSON

Create students.json with:

```
[  
    {"name": "Rahul", "marks": 85},  
    {"name": "Priya", "marks": 92},  
    {"name": "Arjun", "marks": 78}  
]
```

- Read the JSON file.
- Print the student names.
- Calculate the average marks.
- Find the highest scorer.
- Add one new student and save the updated JSON.

CODE:

```
import json  
  
# 1. Create students.json  
students = [  
    {"name": "Rahul", "marks": 85},  
    {"name": "Priya", "marks": 92},  
    {"name": "Arjun", "marks": 78}  
]  
  
with open("students.json", "w") as file:  
    json.dump(students, file, indent=4)  
  
# 2. Read the JSON file  
with open("students.json", "r") as file:  
    students = json.load(file)  
  
# 3. Print student names  
print("Student names:")  
for student in students:  
    print(student["name"])  
  
# 4. Calculate average marks  
total = 0  
for student in students:  
    total = total + student["marks"]  
  
average = total / len(students)
```

```

print("Average marks:", average)

# 5. Find the highest scorer

highest = students[0]

for student in students:

    if student["marks"] > highest["marks"]:

        highest = student

print("Highest scorer:", highest["name"])

print("Highest marks:", highest["marks"])

# 6. Add one new student

new_student = {"name": "Sita", "marks": 88}

students.append(new_student)

# 7. Save the updated JSON

with open("students.json", "w") as file:

    json.dump(students, file, indent=4)

print("New student added and JSON file updated.")

```

OUTPUT:

Student names:

Rahul

Priya

Arjun

Average marks: 85.0

Highest scorer: Priya

Highest marks: 92

New student added and JSON file updated.

6. Part D – Exceptions

Learn to handle bad input instead of allowing the program to crash.

- ValueError – invalid number conversion
- FileNotFoundError – file does not exist
- KeyError – dictionary key does not exist
- ZeroDivisionError – division by zero

Example:

1. ValueError – invalid number conversion

try:

```
age = int(input("Enter age: "))
```

except ValueError:

```
print("Please enter a valid number.")
```

2. FileNotFoundError – file does not exist

try:

```
with open("abc.txt", "r") as file:
```

```
    data = file.read()
```

except FileNotFoundError:

```
print("File does not exist.")
```

3. KeyError – dictionary key does not exist

```
student = {
```

```
    "name": "harshitha",
```

```
    "marks": 85
```

```
}
```

try:

```
print(student["age"])
```

except KeyError: # <-- Added KeyError specifically

```
print("Key does not exist.")
```

4. ZeroDivisionError – division by zero

try:

```
number = int(input("Enter a number: "))
```

```
division = number / 0
```

except ZeroDivisionError:

```
print("A number cannot be divided by zero.")
```

OUTPUT:

Enter age: 20

File does not exist.

Key does not exist.

Enter a number: 19

A number cannot be divided by zero.

7. Part E – Modules

Create utils.py with small reusable functions such as:

- `clean_name(name)`
- `calculate_average(numbers)`
- `is_valid_email(email)`

Create another Python file and import these functions. The goal is to understand how code can be split into reusable modules.

utils.py:

```
def clean_name(name):  
    return name.strip().title()  
  
def calculate_average(numbers):  
    return sum(numbers) / len(numbers)  
  
def is_valid_email(email):  
    return "@" in email and "." in email
```

main.py:

```
from utils import clean_name, calculate_average, is_valid_email  
  
print(clean_name(" yasadwini "))  
print(calculate_average([80, 90, 100]))  
print(is_valid_email("yasaswini@gmail.com"))  
print(is_valid_email("bad-email.com"))
```

OUTPUT:

```
Yasadwini  
90.0  
True  
False
```

8. Part F – Packages

Understand the basic idea of a package: a folder containing related Python modules.

Example structure:

```
project/  
  main.py  
  utils/  
    __init__.py  
    cleaning.py  
    validation.py
```

You do not need to build a complex package. Focus on understanding the structure.

What is a Package?

Answer: A package is simply a folder used to group related Python files (modules) together so your project stays organized.

What is __init__.py?

Answer: It is an empty file that tells Python, "Treat this folder as a special Python package, not just a normal computer folder."

How to use it:

To use the files inside the utils folder from your main.py file, you just combine the folder name and the file name using a dot (.).

Example Code (inside main.py):

```
from utils.cleaning import clean_name  
  
from utils.validation import is_valid_email
```

9. Part G – Quick Debugging

Find the problem, fix it, run the code, and explain the mistake.

Debugging Problem 1

```
file = open("students.csv", "r")  
content = file.read()  
print(content)
```

Task: identify the error and handle/fix it safely.

code:

try:

```
with open("students.csv", "r") as file:  
  
    content = file.read()  
  
    print(content)
```

except FileNotFoundError:

```
    print("students.csv file not found.")
```

output:name,age,marks,city

Rahul,21,85,Hyderabad

Priya,22,92,Bangalore

Arjun,20,78,Chennai

Explanation: We use try/except to handle FileNotFoundError safely

Debugging Problem 2

```
age = int("twenty")
```

Task: identify the error and handle/fix it safely.

code:

```
try:
```

```
    age = int("twenty")
```

```
    print(age)
```

except ValueError:

```
    print("Please enter a number.")
```

output:Please enter a number.

Explanation: We get ValueError because the value is not a valid number.

Debugging Problem 3

```
student = {"name": "Rahul"}
```

```
print(student["marks"])
```

Task: identify the error and handle/fix it safely.

code:

```
student = {"name": "Rahul"}
```

```
try:
```

```
    print(student["marks"])
```

except KeyError:

```
    print("Marks key does not exist.")
```

output:Marks key does not exist

Explanation: We get KeyError because the "marks" key is missing

Debugging Problem 4

```
result = 10 / 0
```

Task: identify the error and handle/fix it safely.

code:

try:

```
    result = 10 / 0
```

```
    print(result)
```

except ZeroDivisionError:

```
    print("Cannot divide a number by zero.")
```

output: Cannot divide a number by zero.

Explanation: We get a ZeroDivisionError because it is mathematically impossible to divide a number by 0. We catch it safely.

10. Part H – Main Hands-on Activity: Data Processing Script

Build a simple data processing script that reads student records, cleans the data, handles bad records, and exports the cleaned result.

Use a CSV file such as:

```
name,age,marks,city
Rahul,21,85,Hyderabad
Priya,22,92,Bangalore
,20,75,Chennai
Arjun,abc,80,Chennai
Sneha,21,95,
Kiran,20,88,Hyderabad
```

Your program should:

- Read the CSV file.
- Check for missing names or cities.
- Handle invalid age/marks values without crashing.
- Clean valid records.
- Ignore or separately report invalid records.
- Calculate the average marks of valid students.
- Find the highest scorer.
- Export valid records into cleaned_students.csv.
- Save invalid records into errors.csv or display them clearly.

Code:

```
import csv

def read_csv(filename):
    with open(filename, "r") as file:
        reader = csv.DictReader(file)
        return list(reader)

def clean_record(record):
    record["name"] = record["name"].strip().title()
    record["city"] = record["city"].strip().title()
    if record["name"] == "" or record["city"] == "":
        return None
    try:
        record["age"] = int(record["age"])
        record["marks"] = int(record["marks"])
    except ValueError:
        return None
    return record

def process_records(records):
    valid_records = []
    invalid_records = []
    for record in records:
        cleaned = clean_record(record.copy())
        if cleaned is not None:
            valid_records.append(cleaned)
        else:
            invalid_records.append(record)
    return valid_records, invalid_records

def export_csv(filename, records):
    if not records:
        return
```

```

with open(filename, "w", newline="") as file:

    writer = csv.DictWriter(file, fieldnames=records[0].keys())

    writer.writeheader()

    writer.writerows(records)

def main():

    records = read_csv("students.csv")

    valid, invalid = process_records(records)

    export_csv("cleaned_students.csv", valid)

    export_csv("errors.csv", invalid)

    total_marks = 0

    highest_marks = 0

    highest_scorer = ""

    for student in valid:

        total_marks += student["marks"]

        if student["marks"] > highest_marks:

            highest_marks = student["marks"]

            highest_scorer = student["name"]

    average = total_marks / len(valid)

    print("Data processing complete.")

    print("Average marks:", average)

    print("Highest scorer:", highest_scorer, "with", highest_marks)

    print("Invalid records saved to errors.csv:", len(invalid))

main()

```

OUTPUT:

Data processing complete.

Average marks: 88.33333333333333

Highest scorer: Priya with 92

Invalid records saved to errors.csv: 3

11. Suggested Program Structure

- read_csv() – read the input file

- `clean_record()` – clean and validate one record
- `process_records()` – process valid records
- `export_csv()` – save cleaned data
- `main()` – control the complete program

Bonus: put reusable cleaning or validation functions into `utils.py`.

Code:

```
import csv

def read_csv(filename):
    with open(filename, "r") as file:
        reader = csv.DictReader(file)
        return list(reader)

def clean_record(record):
    record["name"] = record["name"].strip().title()
    record["city"] = record["city"].strip().title()
    if record["name"] == "" or record["city"] == "":
        return None

    try:
        record["age"] = int(record["age"])
        record["marks"] = int(record["marks"])
    except ValueError:
        return None

    return record

def process_records(records):
    cleaned_records = []

    for record in records:
        cleaned = clean_record(record)

        if cleaned is not None:
            cleaned_records.append(cleaned)

    return cleaned_records

def export_csv(filename, records):
```

```

with open(filename, "w", newline="") as file:

    writer = csv.DictWriter(file, fieldnames=records[0].keys())

    writer.writeheader()

    writer.writerows(records)

def main():

    records = read_csv("students.csv")

    cleaned_records = process_records(records)

    export_csv("cleaned_students.csv", cleaned_records)

    print("Data cleaned and saved successfully.")

main()

```

OUTPUT:

Data cleaned and saved successfully.

12. Part I – Explain in Your Own Words

1. What is file I/O?

Answer: File I/O means reading and writing data in files

2. What is the difference between CSV and JSON?

Answer: CSV stores data in rows and columns. JSON stores data in a structured format.

3. Why do we use try/except?

Answer: We use try/except to handle errors without crashing the program.

4. What is an exception?

Answer: An exception is an error that happens while a Python program is running.

5. What is a Python module?

Answer: A module is a Python file containing reusable code .

6. What is a package?

Answer: A package is a collection of Python modules.

7. Why should we validate data before processing it?

Answer: We validate data to make sure the input is correct and avoid errors.

8. Why is it better to split a large program into functions?

Answer: It is better to split a large program into functions because it makes the code easier to understand, reuse, test, and maintain.

13. Daily Deliverable

Submit your working data processing script.

File name: data_processing_YourName.py

Your script should demonstrate:

- CSV reading
- Data cleaning
- Exception handling
- Data processing
- CSV export
- Functions
- Clear and readable code

14. Student Submission Checklist

- I can read a text file.
- I can read a CSV file.
- I can read and write JSON.
- I can handle common exceptions.
- I understand modules and packages.
- I cleaned invalid data instead of allowing the program to crash.
- I exported the processed data.
- I used functions to organize my program.
- I tested my program with bad input.
- I can explain the important concepts in my own words.

15. After This Assignment, You Should Be Able To

- Work with files using Python.
- Read and write CSV and JSON data.
- Handle common runtime errors safely.
- Create and import simple modules.
- Understand the purpose of packages.
- Clean and validate real-world data.

- Build a small end-to-end data processing script.

Key Principle: Real-world data is rarely perfect. Good programs read it, validate it, handle errors, and produce clean results.